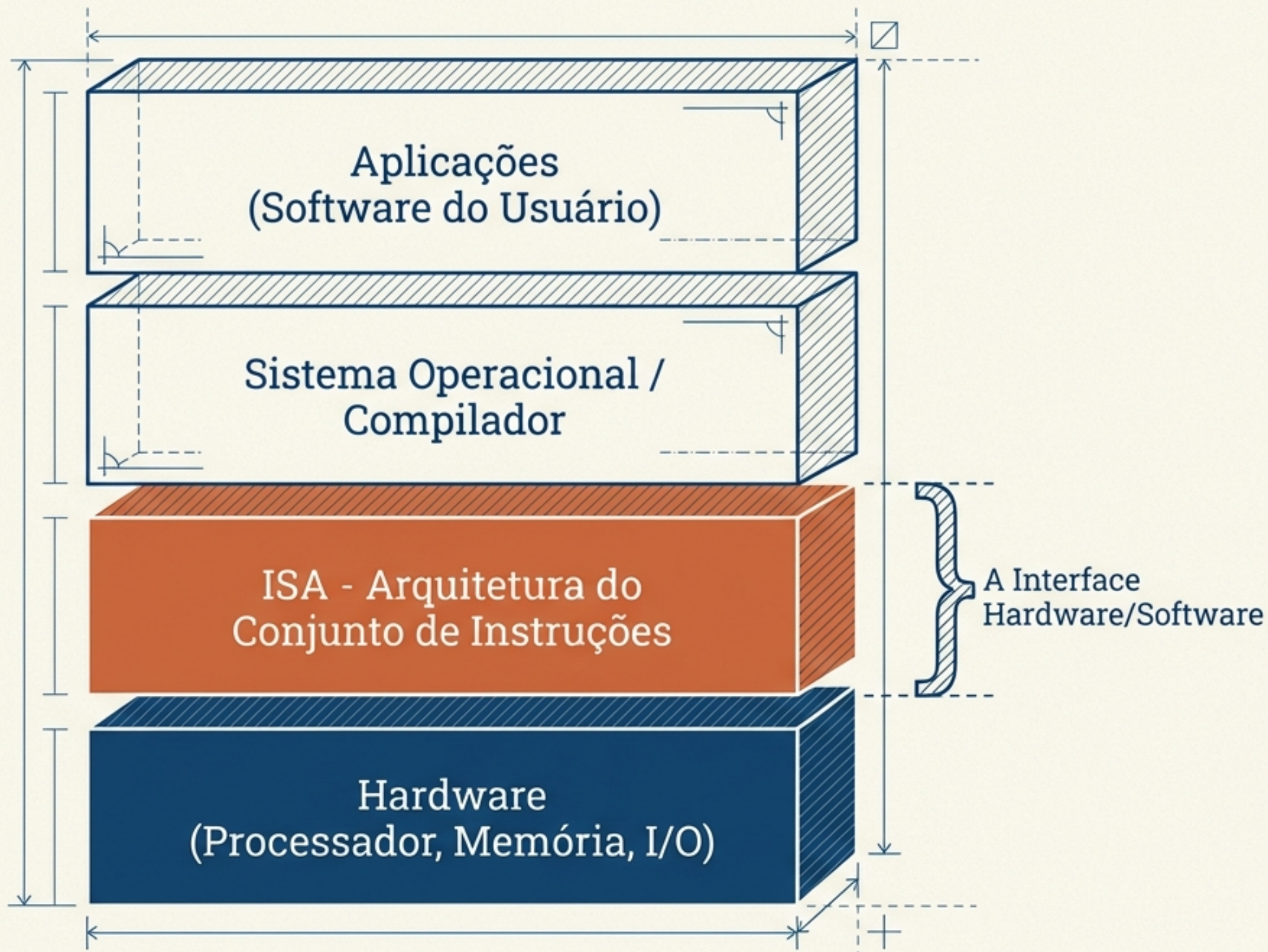


01010111011 add
01010111010 add
01010111011 lw
01010111110 lw
01010111011 sub
01010111101 jr
01010111010 sw
01010111010 add
01010111010 lw
01010111010 jr
01010111010 beq
01010111010 lw
01010111011 sub
01010111010 jr
01010111010 sw
01010101111 lw
01010111011 sub
01010111011 jr
01010111011 sw
01010111011 beq

ARQUITETURA DE COMPUTADORES: A VISÃO DO SOFTWARE

Desmistificando a máquina através
da linguagem Assembly MIPS.





**"A Grande Ilusão:
O hardware é
uma máquina
complexa
disfarçada por
camadas de
software."**

SYSTEM ABSTRACTION LEVEL:
LAYERED INTERFACE

ISA = INSTRUCTION SET ARCHITECTURE

HARDWARE COMPLEXITY MASKED
BY SOFTWARE LAYERS

SWISS EDITORIAL DESIGN STANDARDS:
CLEAN, PRECISE, HIERARCHICAL

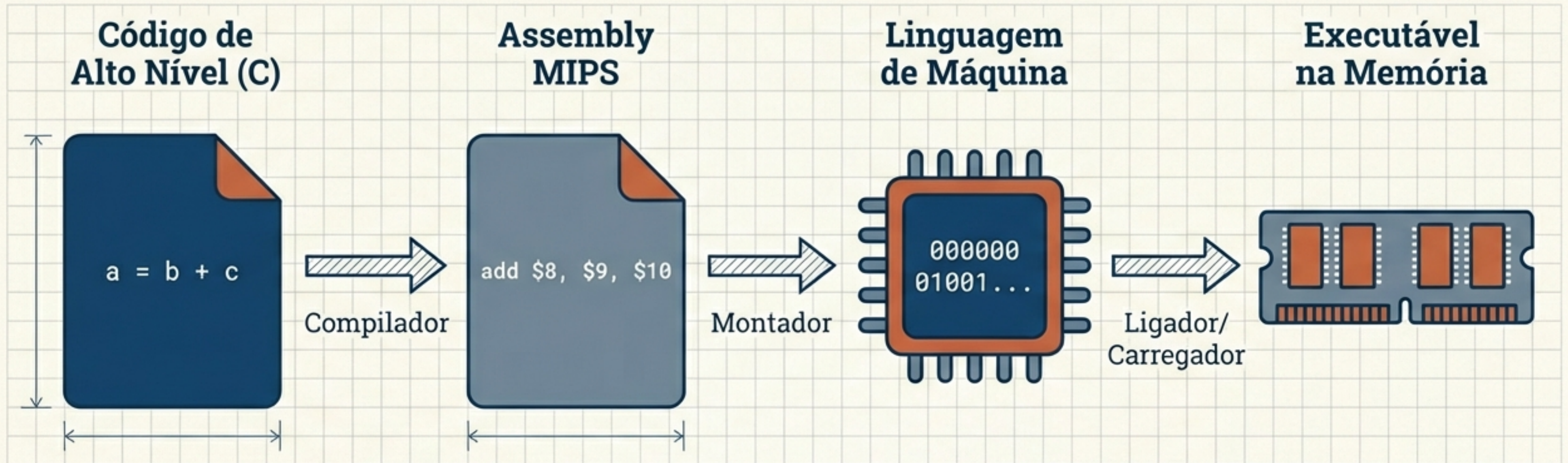
MODERN ENGINEERING BLUEPRINT ELEMENTS:
GRIDS, DIMENSIONS, ANNOTATIONS



Insight: É mais rápido pegar ingredientes na despensa (Registradores) do que ir ao supermercado (Memória).

LEGENDA:
Receita = Programa | Ingredientes = Dados

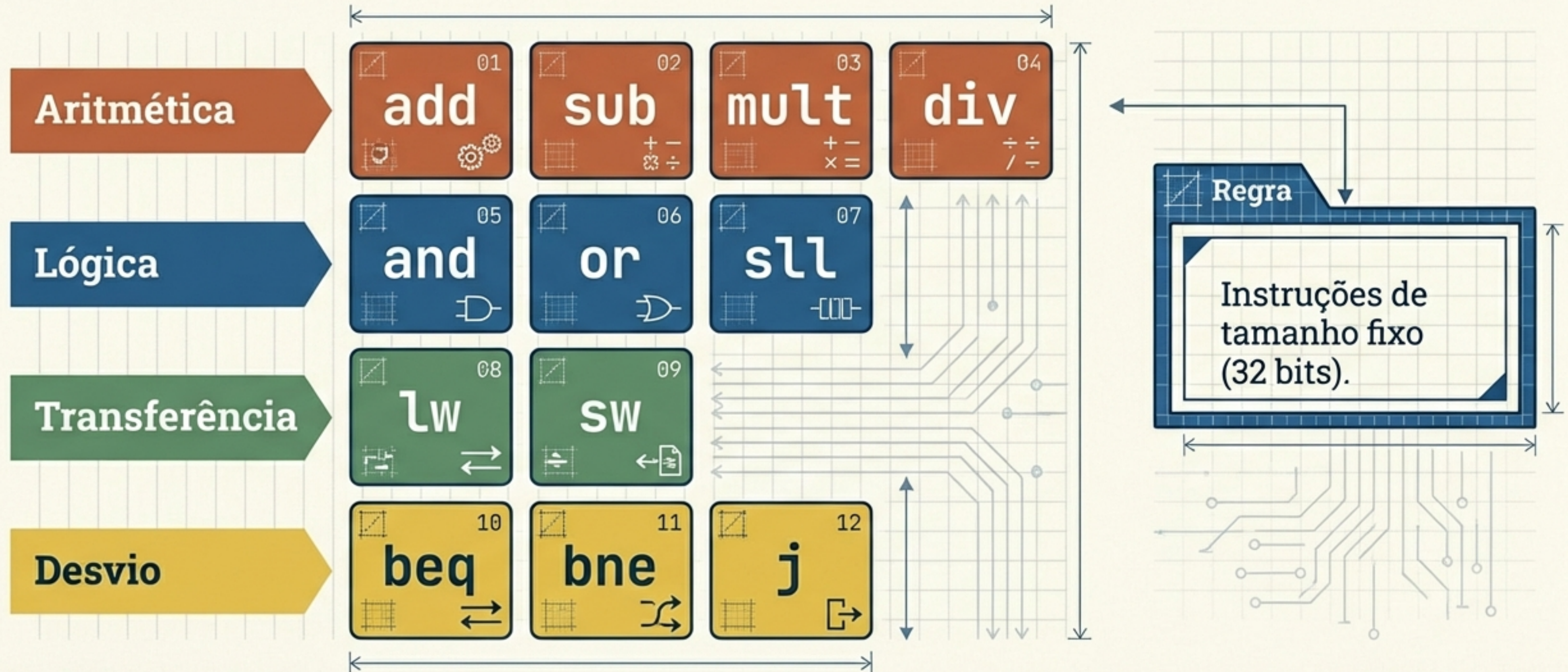
O Caminho da Execução: De Alto Nível ao Hardware



Software de Sistema: A força invisível que gerencia a tradução.

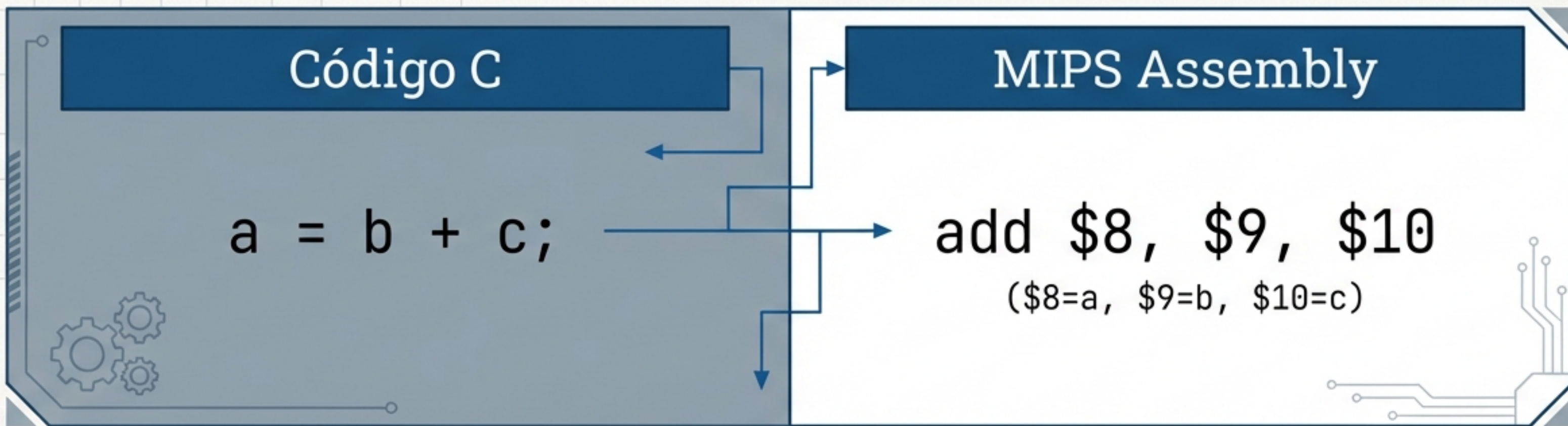
Assembly MIPS: O Vocabulário da Máquina

Vocabulário restrito. Desempenho máximo.



O Banco de Registradores: A 'Despesa' da CPU (MIPS 32 bits)





Soma com sinal: 'add' (gera exceção em Overflow).
Soma sem sinal: 'addu' (ignora Overflow).

Transferência de Dados: Registradores ↔ Memória (MIPS 32 bits)

Registradores (CPU)

Banco de Registradores (\$0 - \$31)

\$s0	32 bit	\$t0	32 bit
\$s1	...	\$s1	...
...	...	...	...
...	...	...	...
\$tn	...	...	...

lw (Load Word) - Compras

Dados (32 bits)

sw (Store Word) - Estoque

Dados (32 bits)

Memória (RAM)

0x10010000	Item 1
0x10010004	Item 2
0x10010008	Item 3
0x10010012	Valor A
0x10010006	Valor B
0x1001000C	Valor B
0x10010040	...
0x1001001B	...
0x10010018	...
0x1001003B	...
0x10010022	...
0x1001002F	...
0x1001002B	...
0x1001002C	...
0x1001003E	...
0x1001003F	...

Endereço = Base + Deslocamento

Base

0x10010000	Item 1
0x10010004	Item 2
0x10010008	Item A
0x1001000C	Item B
0x1001000C	Item B
0x1001000E	...

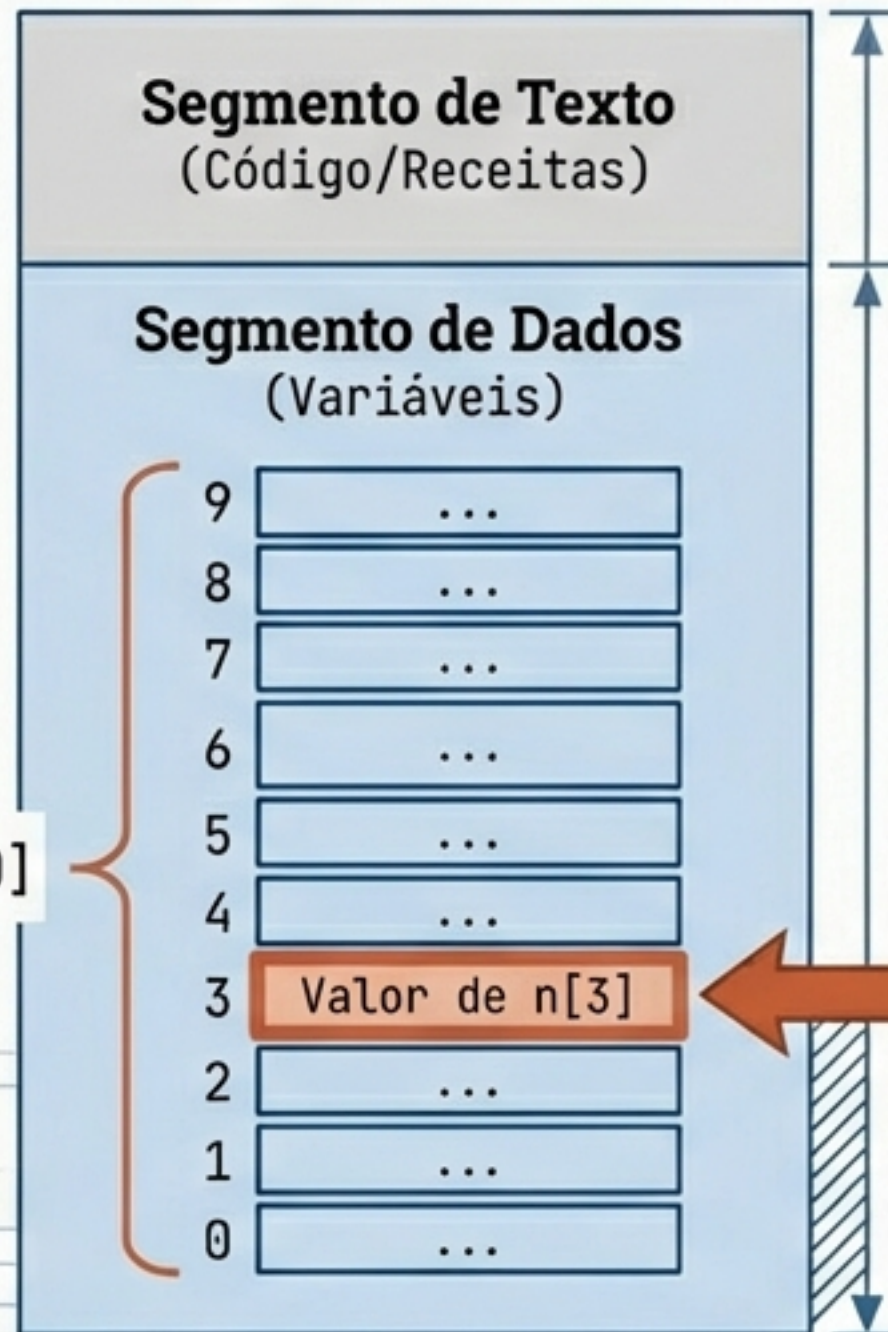
Deslocamento

Item Selecionado

Transferência de Palavras (32 bits).

Acesso a Arrays em Memória (MIPS)

Mapa da Memória

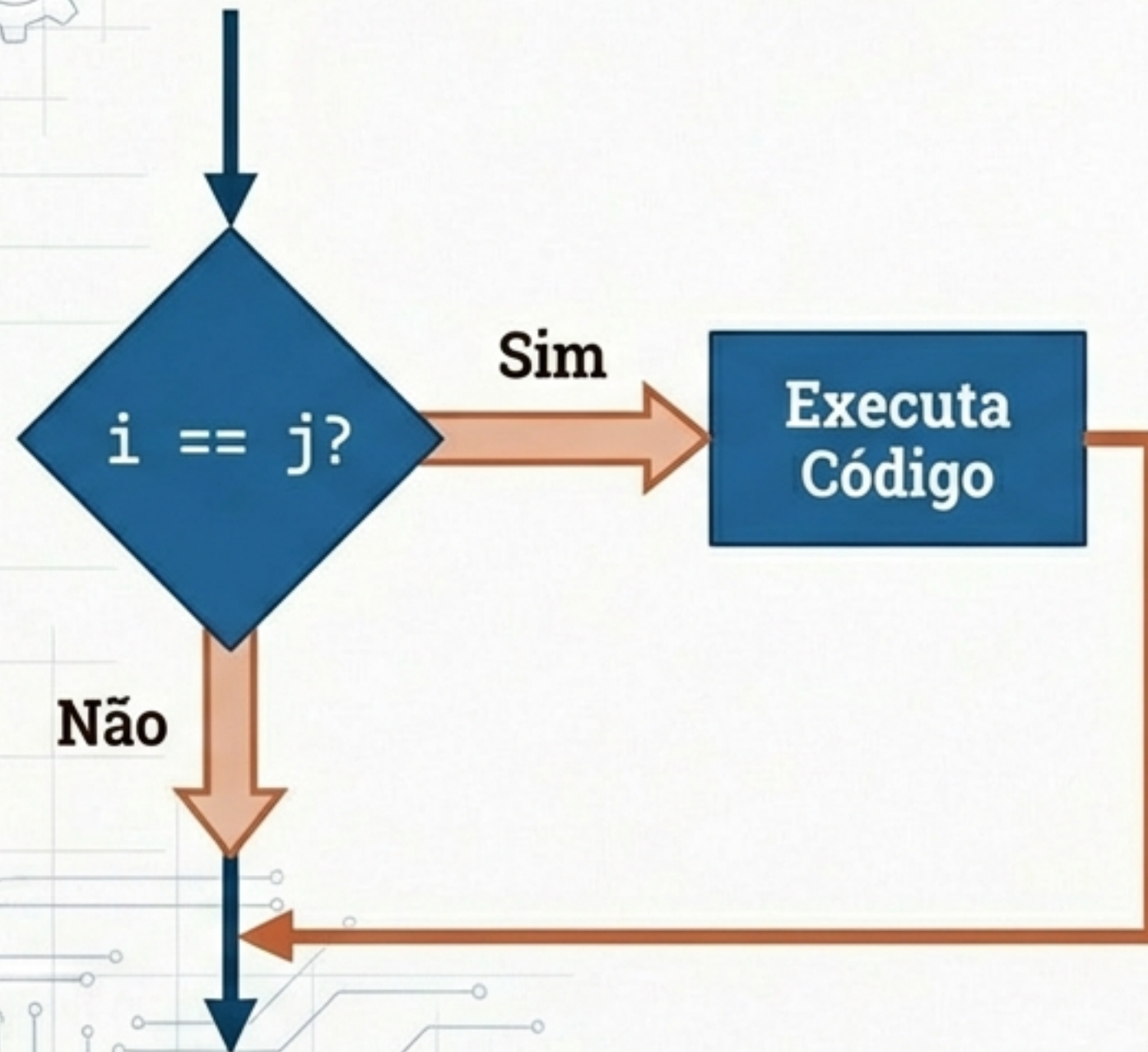


Acesso a $n[3]$:

$$\text{Endereço} = \text{EndereçoBase}(n) + (3 * 4)$$

4 bytes por inteiro

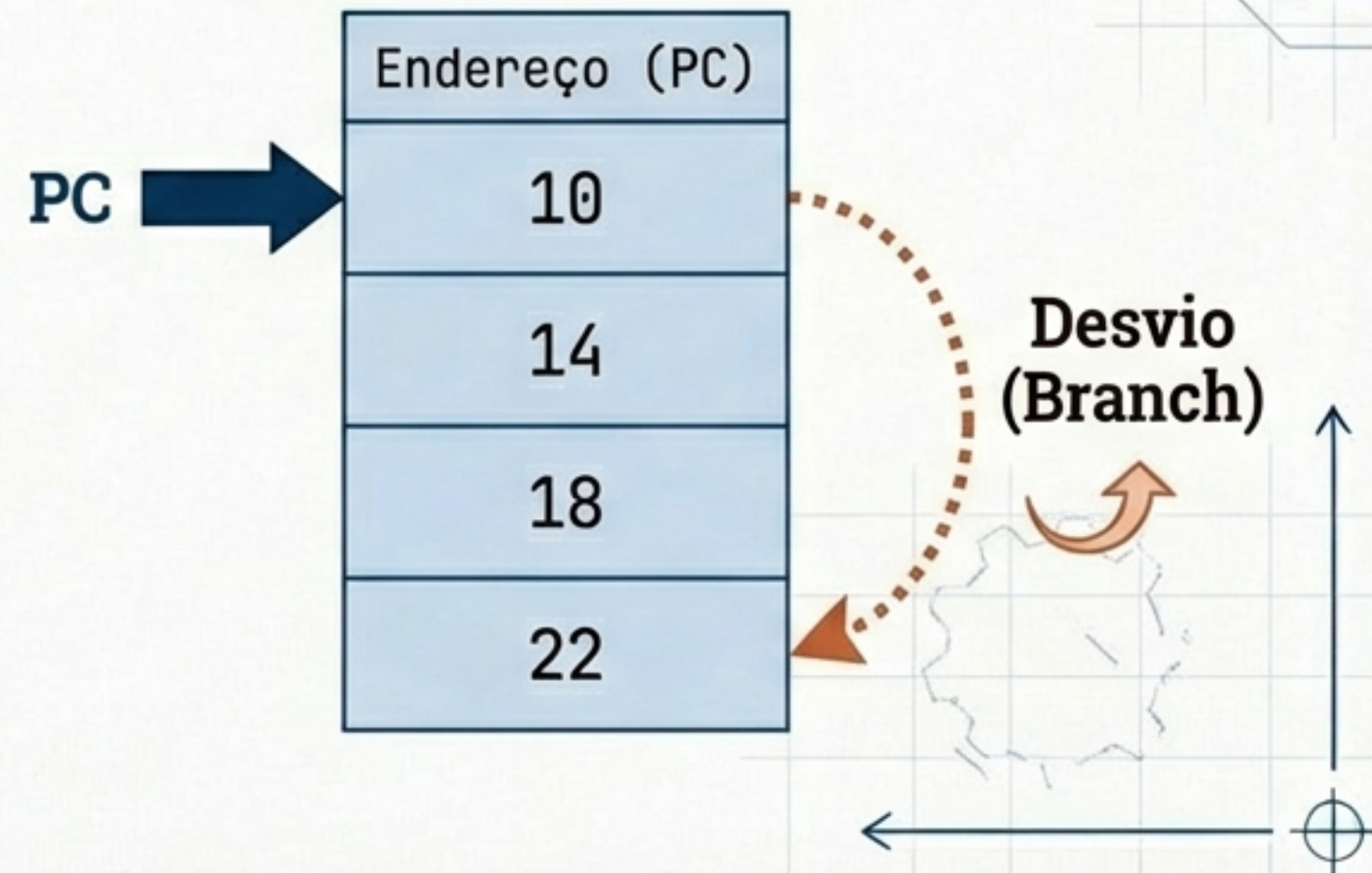
Controle de Fluxo: Desvios e Condicionais (MIPS)



Código e PC

C: `if (i == j) go to Sai;`

MIPS: `beq $8, $9, Sai`



Procedimentos: O Salto e Vínculo

O Chefe pausa a receita, anota onde parou (\$ra),
executa a sub-receita, e **volta**.

Rotina Principal (Main)

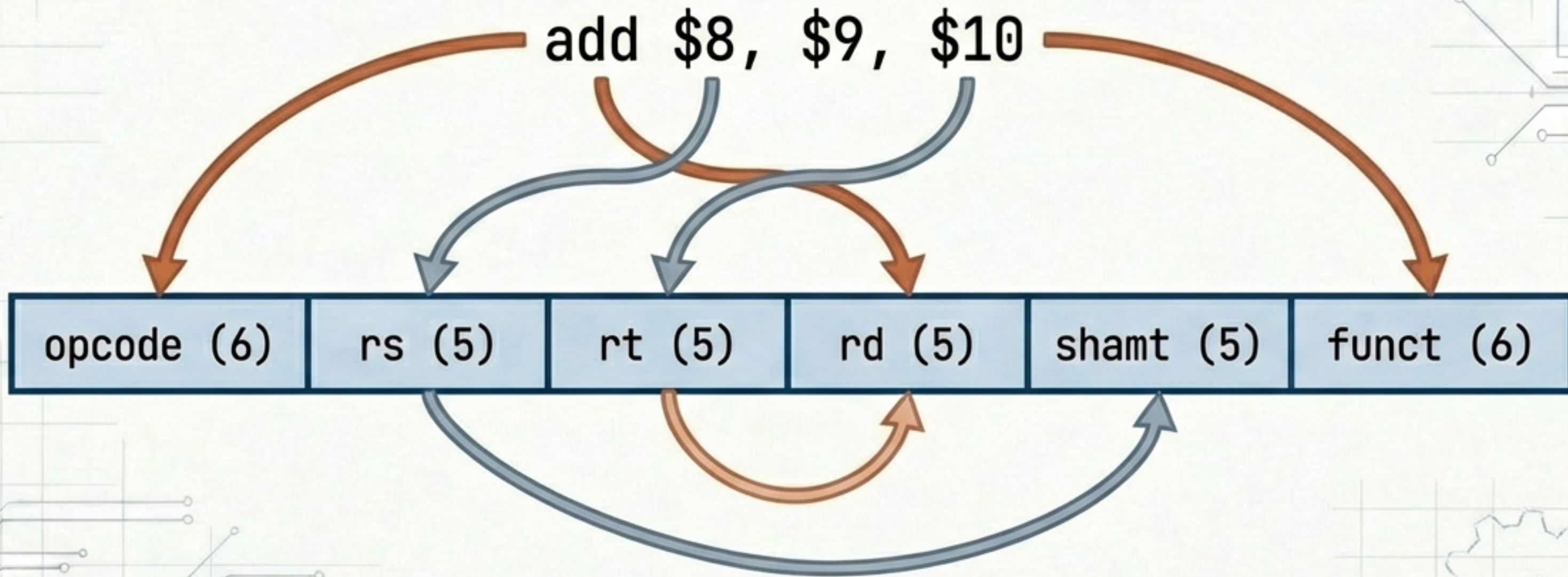
Sub-rotina (Fatorial)

\$ra
(Endereço de Retorno)

jal (Jump and Link)

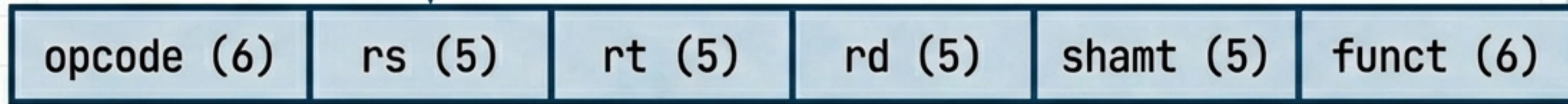
jr \$ra (Jump Register)

Formato R: A Realidade Binária



Formatos I e J: Flexibilidade no Design

R-Format

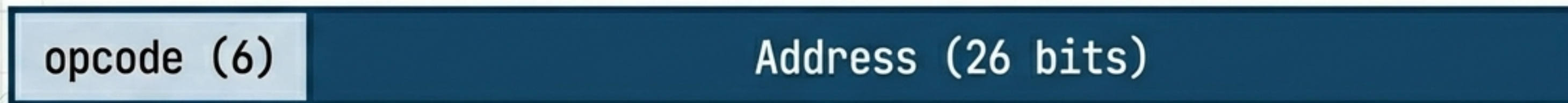


I-Format



Para constantes e acesso à memória (lw, sw, beq).

J-Format



Para saltos longos (j, jal).

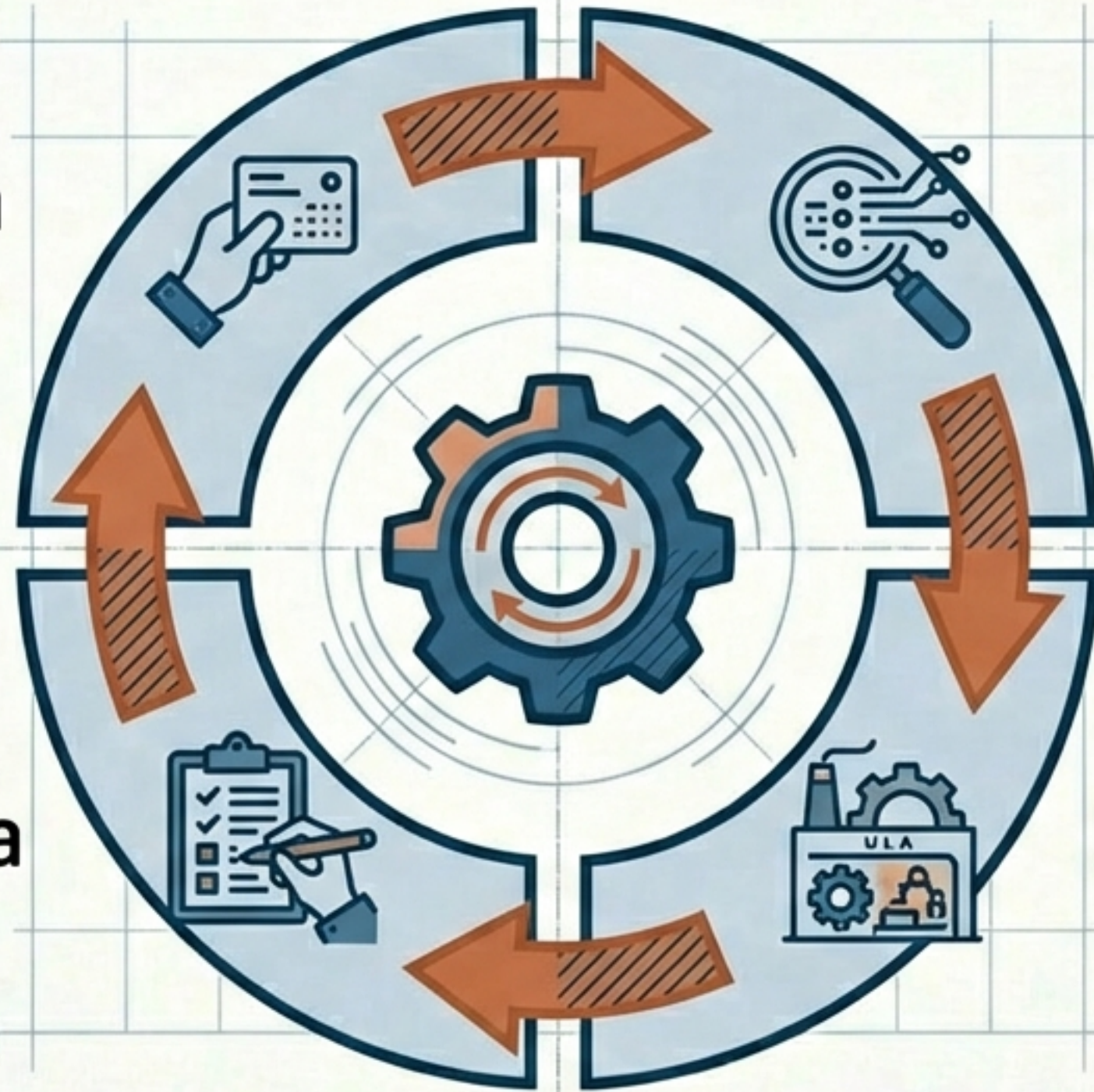
O Ciclo de Vida da Instrução

**Busca
(Fetch)**

Decodificação

Execução

Escrita



Simulado ferramenta SPIM.

Conclusão: A Filosofia MIPS

- ⚙️ 1. Simplicidade favorece a regularidade.
- ⚙️ 2. Menor é mais rápido (Registradores vs Memória).
- ⚙️ 3. Bom design exige bons compromissos.

Próximo Passo: Utilize o simulador SPIM e veja os bits em ação.